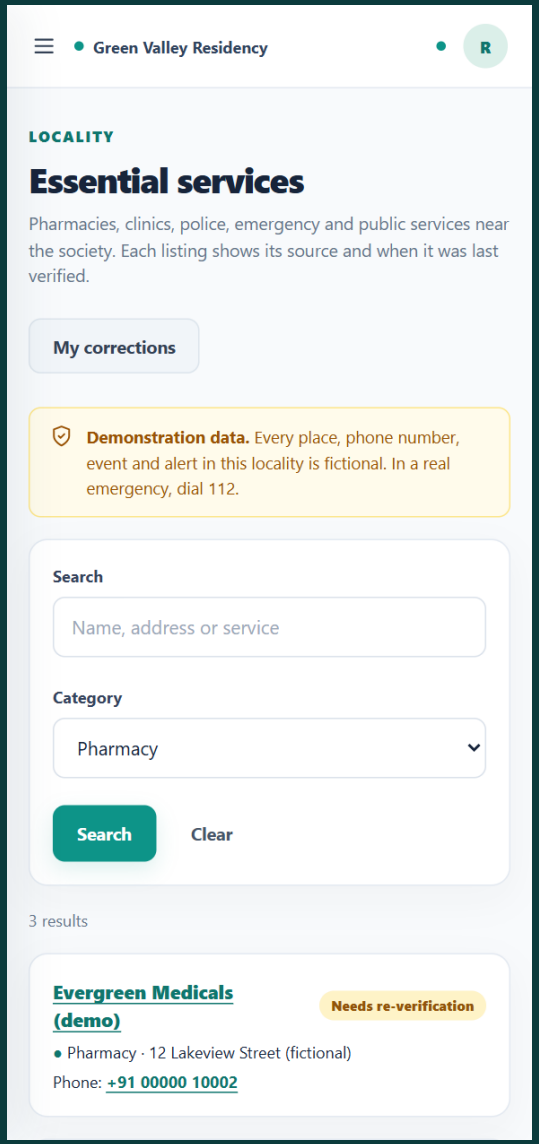
KNOWALL · MODULE 4

Before and after

Locality Intelligence & Community Discovery

What the platform looked like before Module 4, what it looks like now, and what changed in the screens, the API, the database, the roles and the tests.

29 October 2026 · all locality data is fictional demonstration data



1. At a glance

Every number below was measured: the “before” side from a copy of the project with Module 4 removed and from the database backup taken just before the migration; the “after” side from the project as it is now.

54 → 67
frontend pages (+13 locality routes)

119 → 153
API paths in OpenAPI (+34 paths, +37 operations)

40 → 49
PostgreSQL tables (+9 locality tables)

5 → 6
permission roles (+ Locality Coordinator)

235 → 250
backend tests, all passing (+15)

80 → 92
frontend tests, all passing (+12)

0 → 16
verified, sourced service listings (fictional demo)

0 → 3
locality permissions enforced by the server

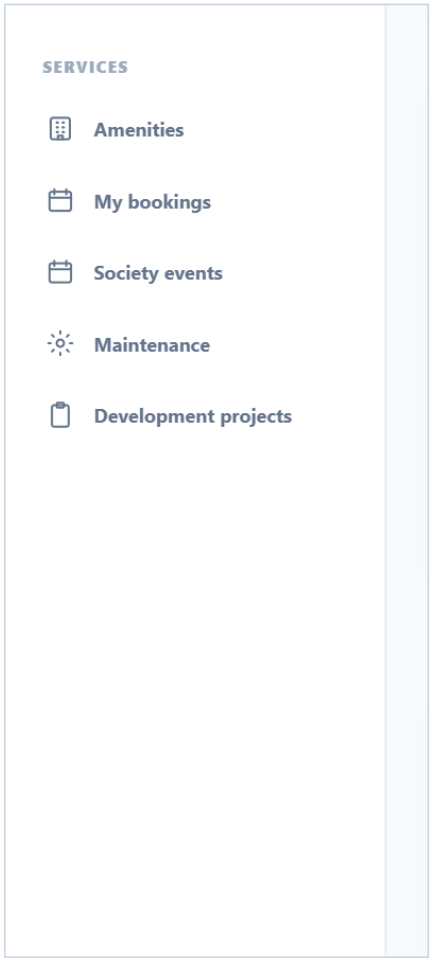
Question	Before Module 4	After Module 4
Where is the nearest pharmacy, and is its number still right?	Not answered by the platform; /locality returned “page not found”.	Essential services directory with search, categories, a coverage map and a verification badge on every listing.
What if a listing is wrong?	No listing to correct and no channel for it.	Residents report outdated information with a reason; a coordinator decides; the resident sees the outcome.
Who keeps neighbourhood information correct?	Nobody; no role existed for it.	A new Locality Coordinator role, separate from the Society Admin.
Neighbourhood events and alerts?	Only society-internal events. A Module 3 “request locality listing” checkbox was stored but nothing used it.	Moderated local events (resident suggestions and opted-in society events) and sourced local alerts that expire.
Can a change be traced?	Not applicable.	Every change is versioned, kept in append-only history and mirrored to the audit log.

All locality data shown is fictional demonstration data.

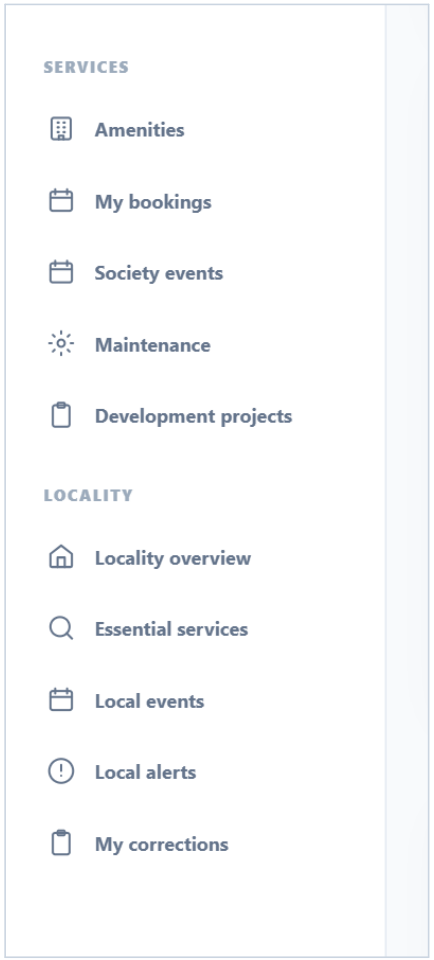
2. Navigation: a new Locality section

Same accounts, same page, sidebars cropped from full-height screenshots. Module 4 adds a LOCALITY group for residents and for the Society Admin; the rest of the menu is unchanged.

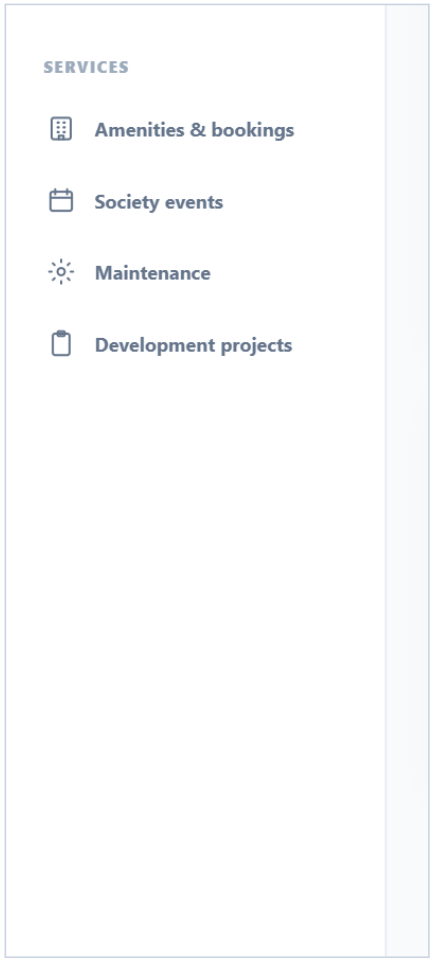
RESIDENT · BEFORE



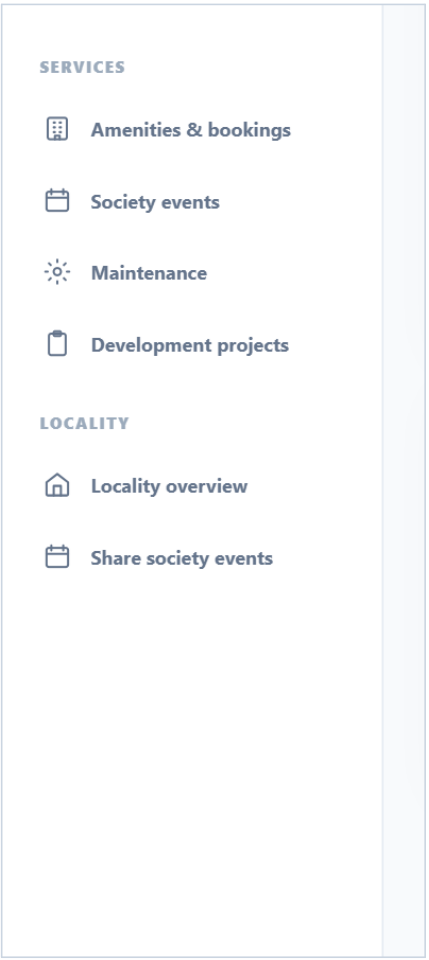
RESIDENT · AFTER



SOCIETY ADMIN · BEFORE



SOCIETY ADMIN · AFTER

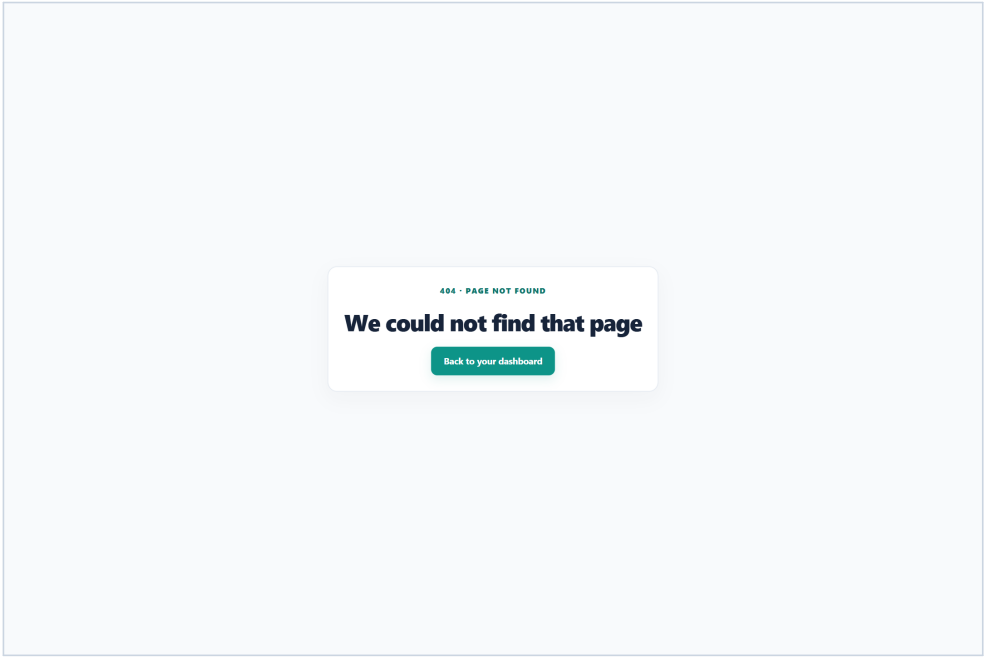


Resident: new items Locality overview, Essential services, Local events, Local alerts and My corrections. **Society Admin:** Locality overview and Share society events only. The Society Admin gets no coordinator desk, by design.

3. Finding an essential service

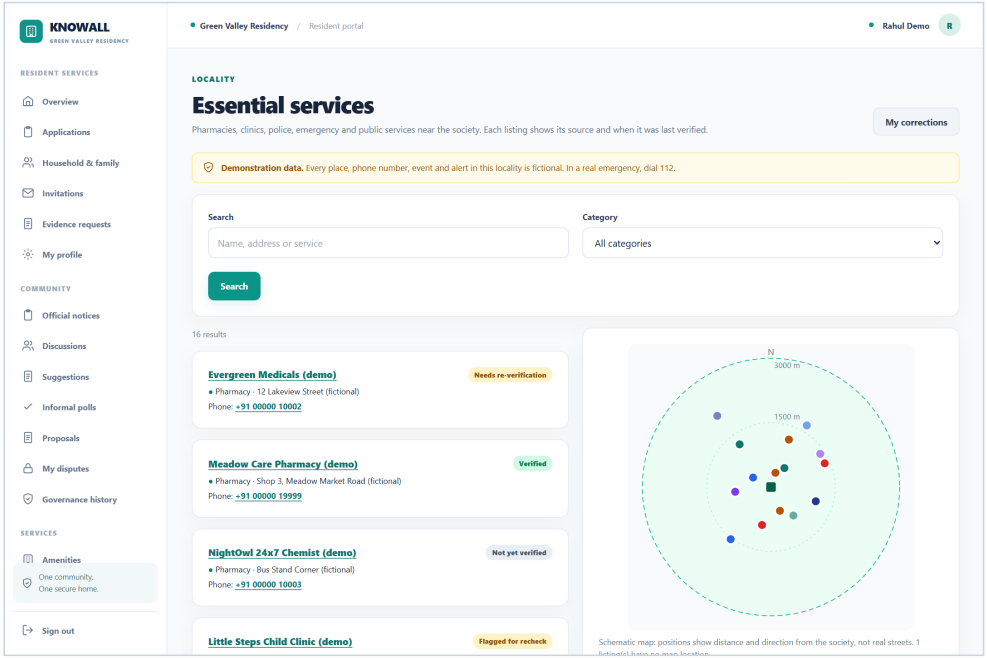
Rahul (resident) opens /locality/places.

BEFORE MODULE 4



The route did not exist, so the application showed its standard “page not found” screen. Residents had no in-app list of nearby pharmacies, clinics, police or emergency services.

AFTER MODULE 4

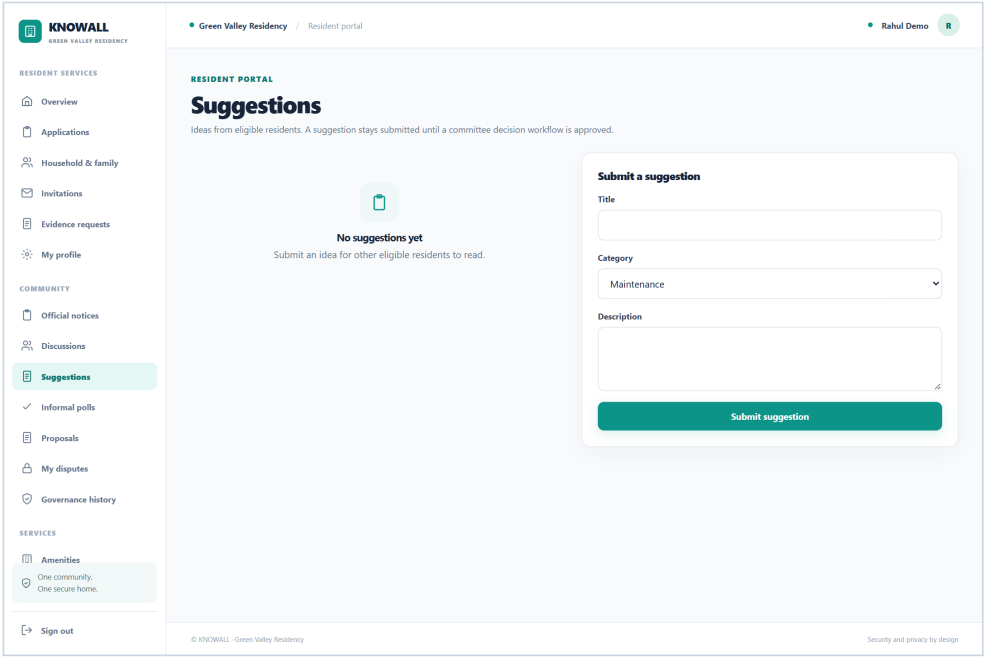


The essential services directory: a fictional-data banner, search and category filter, a verification badge on every listing (Verified, Needs re-verification, Not yet verified, Flagged for recheck) and a schematic map of the coverage area around the society.

4. Reporting outdated information

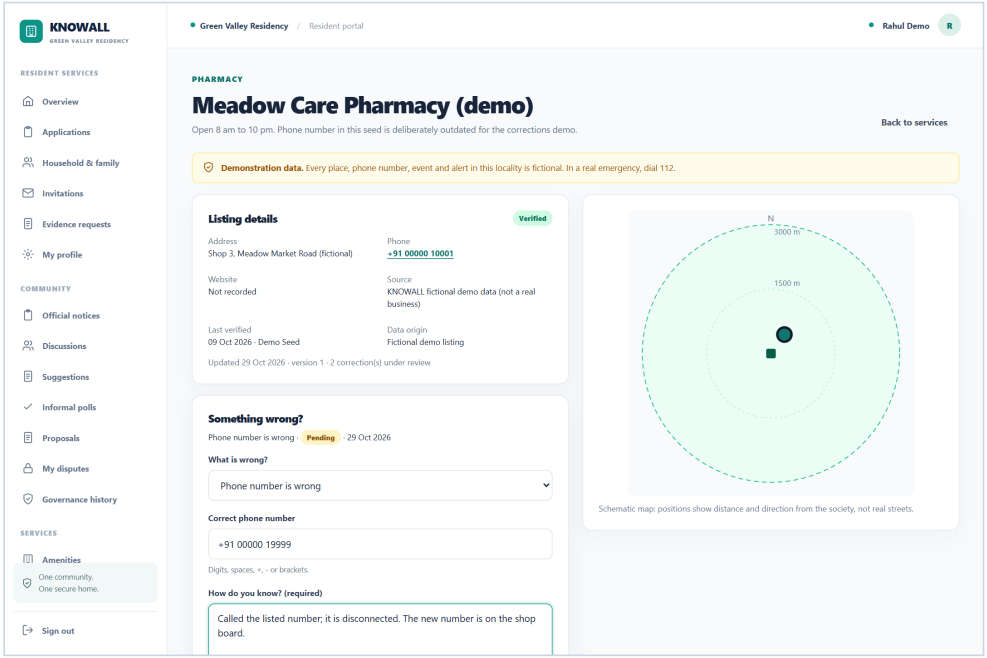
Rahul notices that Meadow Care Pharmacy's number is disconnected.

BEFORE MODULE 4 · closest existing option: Suggestions



There was no listing to correct. The closest existing feature was Suggestions: a general idea board for other eligible residents to read, which stays "submitted" until a committee decision workflow exists. It is not linked to any listing, has no reviewer for neighbourhood information and cannot change anything.

AFTER MODULE 4 · place detail with report form

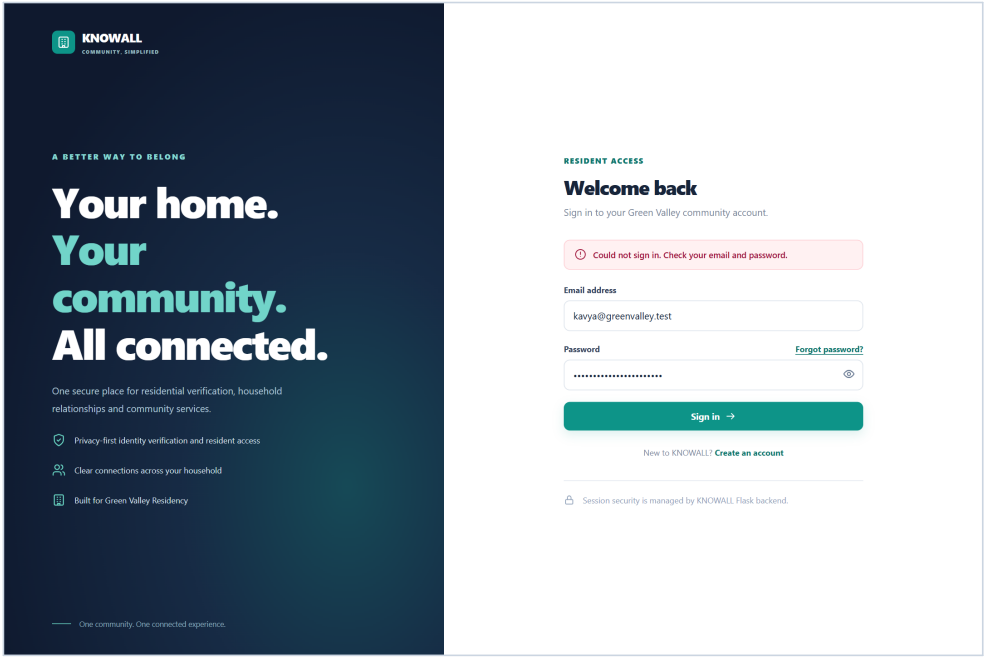


The listing page has a "Report outdated information" form: the issue type, the corrected value, a mandatory reason and an optional evidence link. The report is stored with a snapshot of the value at the time, so the reviewer can see if the listing changed in the meantime.

5. A dedicated Locality Coordinator

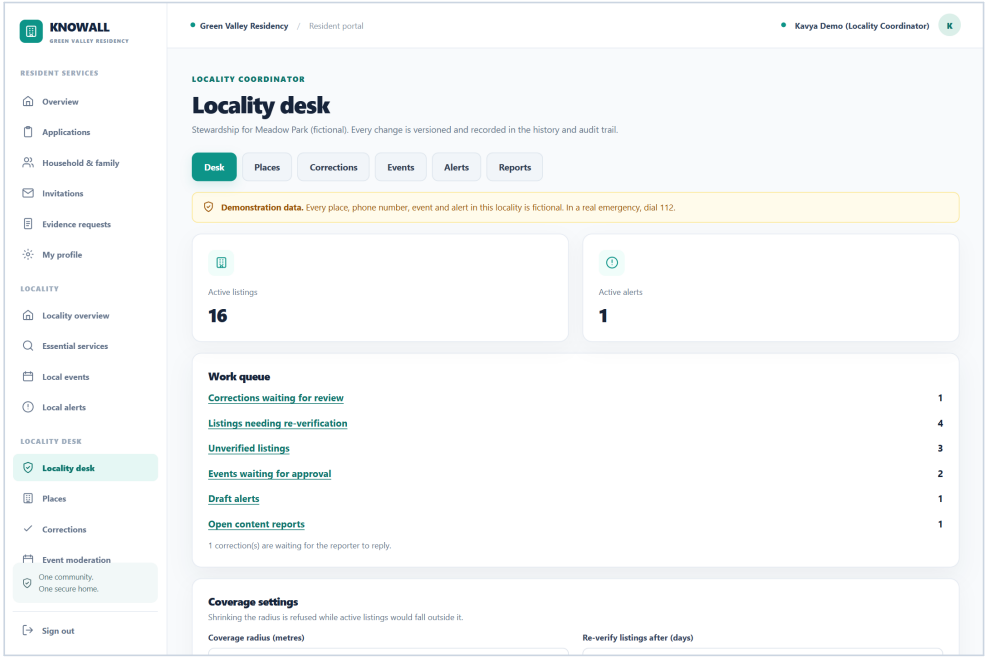
kavya@greenvalley.test signs in through the normal login form.

BEFORE MODULE 4



Sign-in fails with “Could not sign in”. The role LOCALITY_COORDINATOR did not exist (the database only allowed APPLICANT, RESIDENT, COMMITTEE_MEMBER, SOCIETY_ADMIN and PLATFORM_ADMIN), so there was no account and no one responsible for neighbourhood information.

AFTER MODULE 4

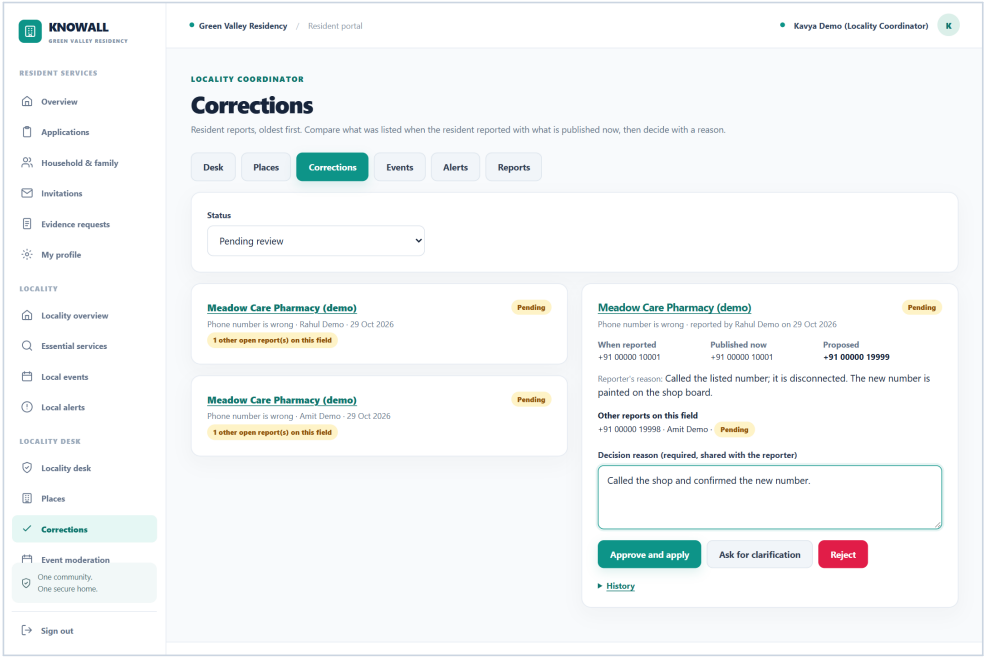


Sign-in lands directly on the Locality desk: active listings and alerts, a work queue (corrections to review, listings needing re-verification, events waiting for approval, draft alerts, open content reports) and the coverage settings.

6. Review and outcome

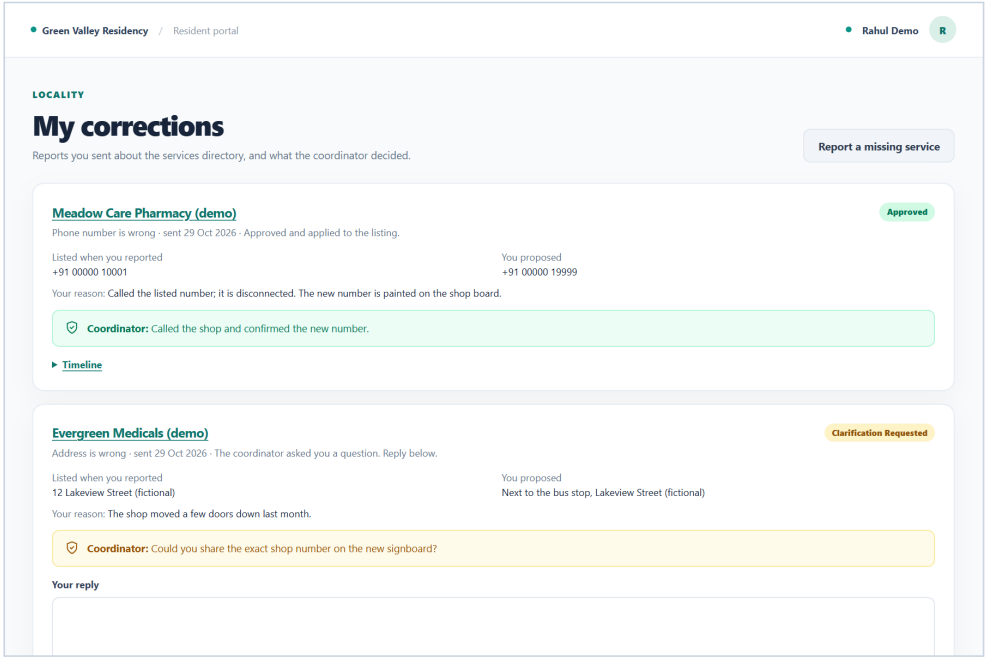
After Module 4 the report is reviewed and the resident sees the result. Before Module 4 there was nothing to review.

AFTER MODULE 4 · coordinator



Coordinator review. The queue flags a competing report from another resident. The panel compares the value when reported, the value published now and the proposed value. A written reason is required to approve, reject or ask for clarification.

AFTER MODULE 4 · resident

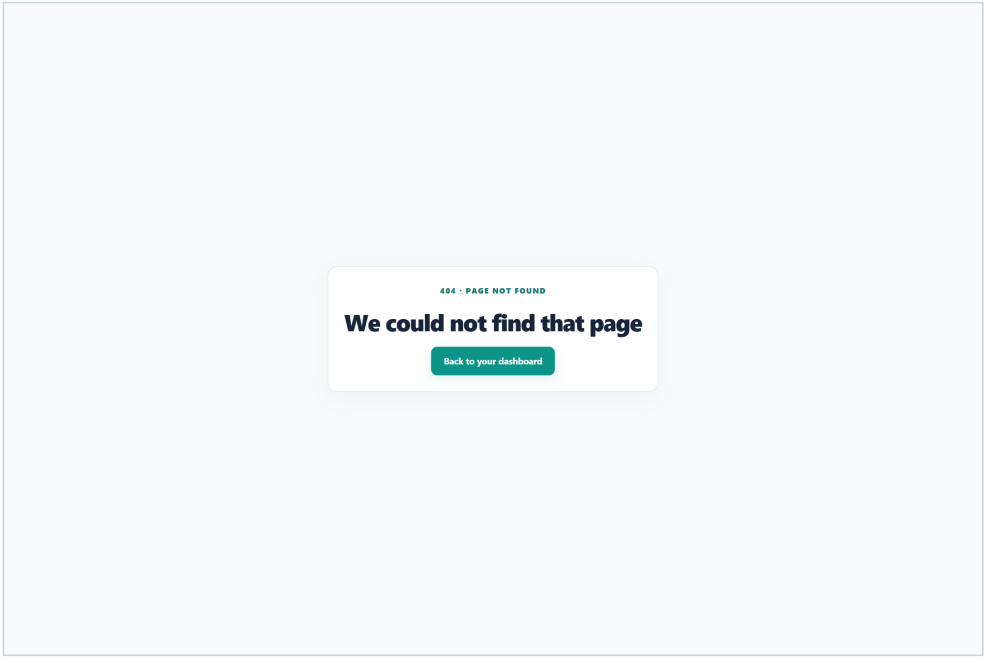


Resident outcome. Rahul sees "Approved" with the coordinator's reason, without the reviewer's name. The listing now shows the corrected number and its public history records "Correction Applied".

7. Sharing society events with the neighbourhood

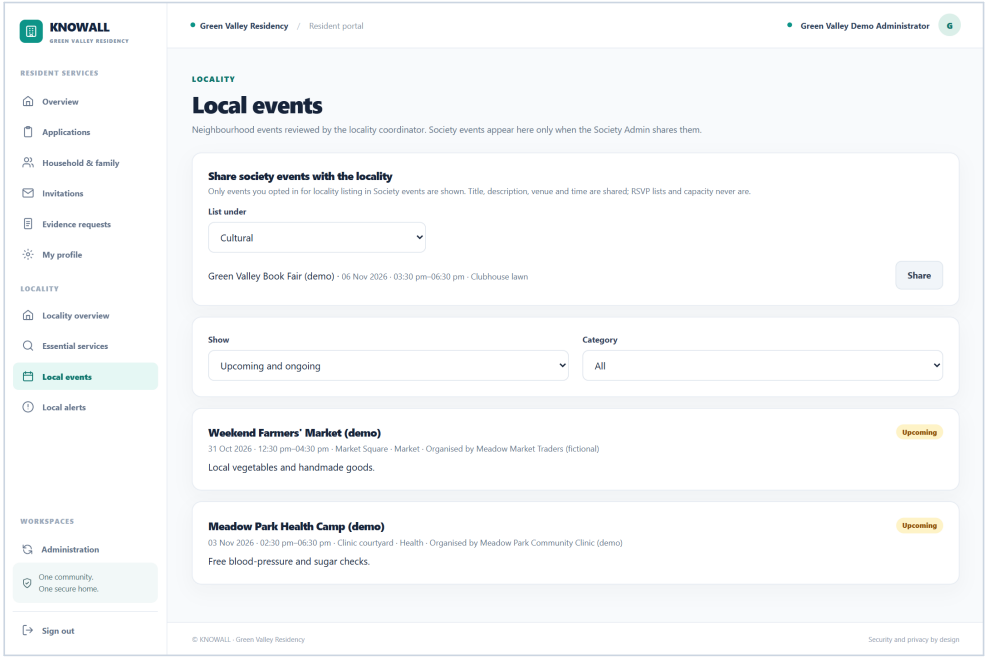
The Society Admin opens /locality/events.

BEFORE MODULE 4



Page not found. Module 3 already let the admin tick “Request locality listing” on a society event, but that flag was only stored; no feature read it, so the event never reached anyone outside the society.

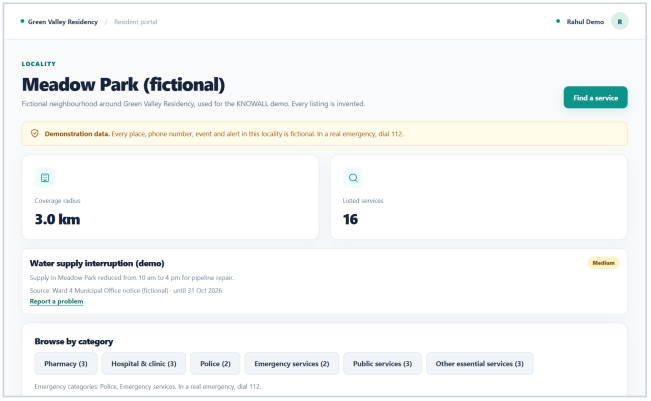
AFTER MODULE 4



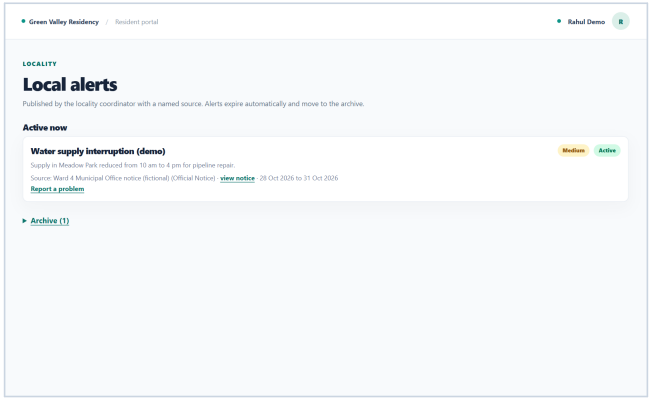
A share section lists only opted-in, published society events. Sharing sends the event to the coordinator for review; title, description, venue and time are shared, RSVP lists and capacity never are. Published local events appear below.

8. Screens that did not exist before

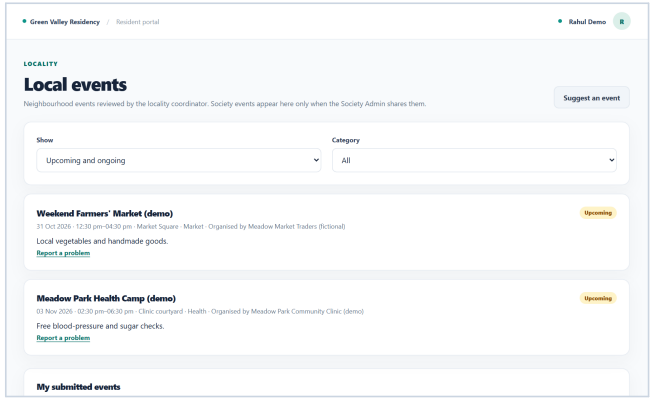
Before Module 4 every one of these routes returned “page not found”.



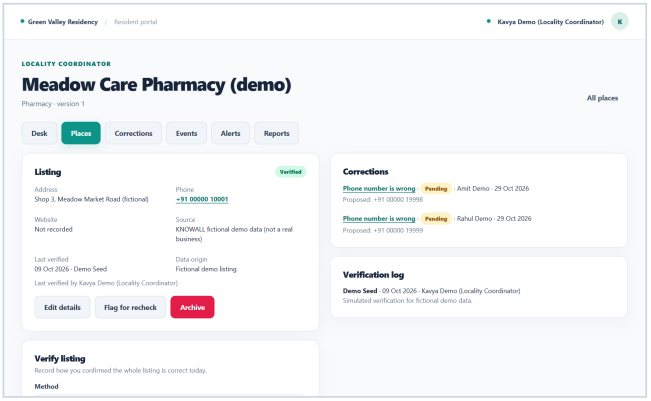
Locality overview: coverage, pinned alert, categories, feed



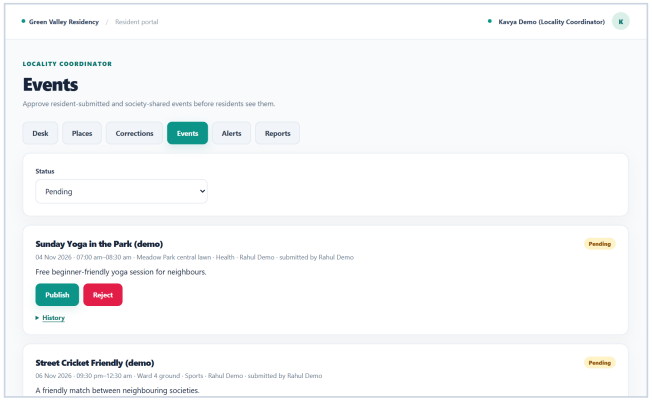
Local alerts with source, severity and expiry



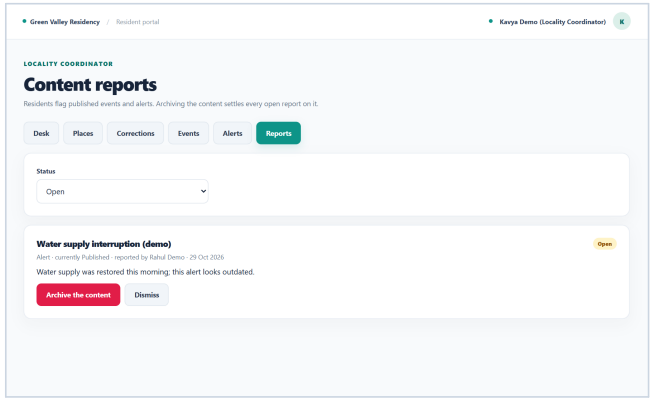
Local events and the resident’s own suggestions



Coordinator place editor: verify, flag, archive, history



Event moderation with a mandatory reason



Content reports: dismiss or archive

9. The API

The same request, signed in as the same resident, against each backend.

BEFORE MODULE 4

```
GET /api/locality/places?category=pharmacy&limit=2

HTTP 404 Not Found

<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
The requested URL was not found on the server. If you entered the URL
manually please check your spelling and try again.
```

Flask had no locality routes, so it answered with its default 404 page. The OpenAPI contract listed 119 paths and 136 operations.

AFTER MODULE 4

```
GET /api/locality/places?category=pharmacy&limit=2

HTTP 200 OK

{
  "places": [
    "(first listing omitted)",
    {
      "name": "Meadow Care Pharmacy (demo)",
      "phone": "+91 00000 19999",
      "category": "Pharmacy",
      "verification_state": "VERIFIED",
      "is_stale": false,
      "last_verified_at": "2026-10-09T15:05:24.495843+00:00",
      "last_verification_method": "DEMO_SEED",
      "source_label": "KNOWALL fictional demo data (not a real business)",
      "data_origin": "FICTIONAL_DEMO",
      "version": 2
    }
  ],
  "pagination": {
    "limit": 2,
    "offset": 0
  }
}
```

JSON with the listing, its category, the computed verification state and staleness, the last verification date and method, the source label and the data origin (here FICTIONAL_DEMO). OpenAPI now lists 153 paths and 173 operations: 34 new locality paths with 37 operations.

10. Database, roles and permissions

Database values come from the PostgreSQL backup taken immediately before the migration and a read-only query of the live development database afterwards.

Item	Before Module 4	After Module 4
Alembic head	d9a4f2c6b871 (Module 2 admin actions)	e5c1a9d3f742 (Module 4 locality, additive)
Tables in public schema	40	49
New tables	—	locality_profiles, place_categories, places, place_verifications, listing_corrections, local_events, local_alerts, locality_content_reports, locality_history
role_assignments.role column	varchar(16)	varchar(30), so LOCALITY_COORDINATOR (20 characters) fits
Allowed roles (check constraint)	APPLICANT, RESIDENT, COMMITTEE_MEMBER, SOCIETY_ADMIN, PLATFORM_ADMIN	the same five + LOCALITY_COORDINATOR
Existing Module 1–3 tables	—	Unchanged apart from the role column above

Role	Locality permissions before	Locality permissions after	What it means now
Resident	locality.listings.suggest (declared in the contract, no feature used it)	locality.read, locality.listings.suggest	Browse the locality; report outdated information; suggest events and missing services.
Society Admin	none	locality.read	Read the locality and share opted-in society events. Cannot open the coordinator desk (403).
Locality Coordinator	role did not exist	locality.read, locality.manage	Maintain and verify listings, review corrections, moderate events, alerts and reports.
Platform Admin	none	none	Unchanged: platform administration does not imply locality powers.

11. Tests and how this comparison was made

Check	Before Module 4	After Module 4
Backend test suite (pytest)	235 passed	250 passed (15 new locality tests)
Frontend test suite (node --test)	80 passed	92 passed (12 new locality tests)
TypeScript type check	clean	clean
Frontend pages (page.tsx files)	54	67

How the “before” state was produced

The project is not a git repository, so there was no earlier commit to check out. Instead a scratch copy of the backend and frontend was made and exactly the Module 4 additions were removed from it:

- deleted: `app/modules/locality/`, migration `e5c1a9d3f742`, `seed_locality.py`, the locality tests, `src/app/locality/`, `src/features/locality/` and `lib/locality-*.ts`;
- reverted the Module 4 lines in the shared files: blueprint registration, the locality permissions, the new role, the sidebar groups, the coordinator login redirect, and `openapi.json` (restored from the copy saved before Module 4);
- a file-by-file comparison confirmed the copy differs from the current project only by those files.

The test counts above were obtained by running both suites on that copy (235 and 80, exactly the current totals minus the Module 4 tests). Screenshots were taken from the copy (Next.js on port 3100, Flask on port 5002, fresh SQLite database with the standard Green Valley seed) and from the current application (Flask on port 5001 with a scratch SQLite database). The real development database was never written to; only the backup and a read-only query were used for section 10.

Some screenshots in sections 4, 6 and 8 come from the Module 4 technical report, captured the same way.